Basic Skill in Brief (BSB)

1.1 Variables, Loops, Conditions

- What are they?
  - Variable → A named box that stores a value.
  - Condition (if/else) → Decide what to do based on a rule.
  - Loop (for/while) → Repeat something multiple times.
- Why important in apps?
  - Store data like username, score, token.
  - Show different screens depending on login status.
  - Loop through lists like products, chats, notifications.
- Example (Kotlin – Android)

kotlin Copy code

```kotlin
var username: String = "Shashwat"
val maxScore: Int = 100  // val = cannot change

if (username.isNotEmpty()) {
    println("Welcome, $username")
} else {
    println("Please enter username")
}

for (i in 1..5) {
    println("Tap count: $i")
}
```

In an app:

- username might come from a text field.
- Loop shows items in a RecyclerView / List.

1.2 OOP (Object-Oriented Programming)

- Main ideas
  - Class: Blueprint (e.g. User, Product, Message)

- Object: Real thing created from class.
- Encapsulation: Hiding internal details.
- Inheritance: One class extends another.
- Polymorphism: Same function name, different behavior.

🔹 Why in apps?

- Represent real-world things: users, orders, screens.
- Clean structure → easier to maintain.
- Reuse code across screens/platforms.

🔹 Example (Dart – Flutter)

dart Copy code

```dart
class User {
  String name;
  String email;

  User(this.name, this.email);

  void greet() {
    print("Hello, $name");
  }
}

void main() {
  User u = User("Shashwat", "s@example.com");
  u.greet(); // Hello, Shashwat
}
```

In an app, you might have a User class with fields like id, token, profilePicUrl.

1.3 APIs & JSON

🔹 API

- API (Application Programming Interface) → A *bridge* between your app and a server.
- Example: Weather app → calls an API → gets today's temperature.

🔹 JSON

- JSON = JavaScript Object Notation
- A text format used to send data like:

```json
{
  "name": "Shashwat",
  "age": 21,
  "isPremium": true
}
```

🔹 Why important?

- Almost every serious app uses APIs and JSON:
  - Login, signup
  - Fetch products, posts, videos
  - Send messages, payments, etc.

1.4 Asynchronous Programming

🔹 Problem

APIs, file reading, database calls → take time.
If you block the main thread, app freezes.

🔹 Async solution

Let slow tasks run in background, and when done, update UI.

🔹 Terminology

- Future / Promise (Dart, JS)
- suspend / coroutines (Kotlin)
- async/await (Swift, JS, Dart, etc.)
- Callbacks (older style)

🔹 Example (Dart – Flutter)

```dart
Future<String> fetchData() async {
  await Future.delayed(Duration(seconds: 2));
  return "Data from server";
}

void main() async {
  print("Loading...");
  String result = await fetchData();
  print(result);
}
```

In apps, await is what prevents UI from freezing and still allows results when done.

2️⃣ Mobile UI/UX Concepts

2.1 Navigation

🔹 What is it?

Moving between screens/pages in the app.

- Login → Home
- Home → Profile
- Product → Checkout

🔹 Platform mapping

- Android: Intent, NavController, navigation graph
- iOS: UINavigationController, pushViewController
- Flutter: Navigator.push, go_router, auto_route
- React Native: react-navigation

🔹 Example (Flutter)

```dart
Navigator.push(
  context,
  MaterialPageRoute(builder: (context) => ProfileScreen()),
);
```

2.2 Layouts

🔹 Purpose

Arranging UI elements on the screen.

- Horizontal / vertical
- Grids
- Scrollable lists

🔹 Platform mapping

- Android XML: LinearLayout, ConstraintLayout, RecyclerView
- Jetpack Compose: Row, Column, LazyColumn
- iOS UIKit: UIStackView, AutoLayout constraints
- SwiftUI: VStack, HStack, List
- Flutter: Row, Column, ListView, GridView
- React Native: View, ScrollView, FlatList

🔹 Example (Flutter simple layout)

dart Copy code

```
Column(
  children: [
    Text("Welcome"),
    ElevatedButton(onPressed: () {}, child: Text("Get Started")),
  ],
)
```

2.3 Responsive Design

🔹 What is it?

Design that works on:

- Different screen sizes
- Phones & tablets
- Portrait / landscape

🔹 Techniques

- Use percentage-based width/height

- Use flexible layouts (ConstraintLayout, weight, Expanded)
- Use breakpoints or media queries

🔹 Example (Flutter)

```dart
double width = MediaQuery.of(context).size.width;

Container(
  width: width * 0.9,   // 90% of screen width
  child: Text("Responsive box"),
);
```

2.4 Accessibility

🔹 Goal

Make app usable by:

- People with visual issues
- People using screen readers
- People who can't tap small buttons

🔹 Practices

- Sufficient contrast
- Larger tap areas (min ~44x44 points)
- Proper labels for screen readers
- Don't rely only on color to show status

🔹 Example

- Android: android:contentDescription
- iOS: accessibilityLabel
- Flutter: Semantics widget

```dart
Semantics(
  label: 'Profile picture of Shashwat',
  child: CircleAvatar(...),
);
```

3️⃣ App Architecture

3.1 MVC (Model–View–Controller)

- Model: Data / business logic (User, Product).
- View: UI (screens).
- Controller: Connects Model and View.

Used more in older iOS and web, still good to know.

3.2 MVVM (Model–View–ViewModel)

Most popular in Android, iOS, Flutter now.

- Model: Data layer.
- View: UI (Activity/Fragment/Widget/ViewController).
- ViewModel: Holds UI logic + state (no UI code).

🔹 Flow

User clicks button → View calls ViewModel → ViewModel updates state → View observes and redraws.

🔹 Example (rough idea)

kotlin Copy code

```kotlin
class LoginViewModel : ViewModel() {
    val loginState = MutableLiveData<Boolean>()

    fun login(email: String, password: String) {
        // call API, etc.
        loginState.value = true
    }
}
```

Activity observes loginState and navigates on success.

3.3 Clean Architecture (Layers)

Typical layers:

1. Presentation → UI + ViewModel

2. Domain → Use cases (business logic)
3. Data → Repositories, APIs, DB

🔹 Why?

- Easier to test
- Swap API/DB without touching UI
- Large projects stay manageable

3.4 State Management (Concept)

"State" = data that affects what you see on screen.

Examples:

- Is user logged in?
- Cart items
- Current tab index
- Is loading? (spinner)

State management = how you store, change, and share that data across the app in a clean way.

🌐 Networking

4.1 REST APIs

🔹 Concepts

- REST: Style for web services.
- Uses HTTP methods:
    - GET → fetch data
    - POST → create
    - PUT/PATCH → update
    - DELETE → remove

Example:
GET /users/1 → details of user 1.

4.2 Calling APIs

Android: Retrofit

```kotlin
interface ApiService {
    @GET("users")
    suspend fun getUsers(): List<User>
}
```

iOS: Alamofire / URLSession

```swift
AF.request("https://api.example.com/users").response { response in
    // handle data
}
```

Flutter: http package / Dio

```dart
final response = await http.get(Uri.parse("https://api.example.com/users"));
```

4.3 JSON Parsing

- Idea

Convert JSON string → objects (model classes).

Example JSON

```json
{
  "id": 1,
  "name": "Shashwat"
}
```

Kotlin data class

```kotlin
data class User(val id: Int, val name: String)
```

Dart model

dart

Copy code

```dart
class User {
  final int id;
  final String name;

  User({required this.id, required this.name});

  factory User.fromJson(Map<String, dynamic> json) {
    return User(
      id: json['id'],
      name: json['name'],
    );
  }
}
```

5️⃣ Databases

5.1 Local Databases

Used for:

- Offline mode
- Caching
- Storing user preferences

🔹 SQLite

- C-based, default DB in Android, iOS, etc.
- Raw SQL queries.

🔹 Room (Android, on top of SQLite)

kotlin

Copy code

```kotlin
@Entity
data class Note(
  @PrimaryKey(autoGenerate = true) val id: Int,
  val text: String
)
```

```
@Dao
interface NoteDao {
  @Insert
  suspend fun insertNote(note: Note)

  @Query("SELECT * FROM Note")
  suspend fun getAllNotes(): List<Note>
}
```

- 🔹 Core Data (iOS)
  - Apple's framework for object graph & persistence.
- 🔹 Hive / Drift (Flutter)
  - Hive: Fast key-value DB.
  - Drift: SQLite wrapper with nice APIs.

5.2 Cloud Databases

- 🔹 Firebase
  - Realtime Database or Firestore
  - Sync data between users in real time.
  - Great for chat apps, live dashboards.
- 🔹 Supabase
  - Open-source alternative to Firebase.
  - Postgres DB + auth + storage.

Example (Firestore in Flutter)

dart Copy code

```
FirebaseFirestore.instance.collection('users').add({
  'name': 'Shashwat',
  'createdAt': FieldValue.serverTimestamp(),
});
```

6️⃣ App Lifecycle

6.1 Activity Lifecycle (Android)

Common callbacks:

- onCreate() → screen created
- onStart() → visible
- onResume() → interacting
- onPause() → partially hidden
- onStop() → not visible
- onDestroy() → being destroyed

Why important?

- Start/stop animations
- Save data when leaving screen
- Cancel API calls when screen goes away

6.2 Widget Lifecycle (Flutter)

- initState() → once when widget created.
- build() → every time UI updates.
- didUpdateWidget() → when parent passes new data.
- dispose() → clean up (controllers, streams).

Example

dart Copy code

```dart
class MyWidget extends State<MyWidget> {
  @override
  void initState() {
    super.initState();
    // init controllers, fetch data
  }

  @override
  void dispose() {
    // dispose controllers
    super.dispose();
  }
}
```

6.3 ViewController Lifecycle (iOS)

Key events:

- viewDidLoad() → initial setup.
- viewWillAppear() → about to be visible.
- viewDidAppear() → fully visible.
- viewWillDisappear() → going away.
- viewDidDisappear() → gone.

Use these to:

- Start/stop timers
- Register/unregister observers
- Refresh data each time the view appears

7️⃣ State Management (In Practice)

7.1 Provider (Flutter)

Simple and widely used.

Idea

- Store state in a ChangeNotifier.
- UI "listens" and rebuilds when state changes.

Example

```dart
class CounterProvider extends ChangeNotifier {
  int count = 0;

  void increment() {
    count++;
    notifyListeners();
  }
}
```

In UI:

```dart
Consumer<CounterProvider>(
  builder: (context, provider, child) {
    return Text(provider.count.toString());
  },
);
```

7.2 Riverpod / Bloc (Flutter)

- Riverpod: Type-safe, testable, more advanced Provider.
- Bloc: Event → Bloc → State pattern.

Example idea for Bloc

- Event: LoginButtonPressed
- Bloc: calls repository, updates state
- State: LoginLoading, LoginSuccess, LoginFailure

Good for large apps.

7.3 ViewModel (Android / iOS / Flutter)

You usually have:

- View = Activity/Fragment/Widget
- ViewModel = holds state & logic
- Repository = talks to API/DB

Example (Android ViewModel):

```kotlin
class HomeViewModel(private val repo: UserRepository) : ViewModel() {
    val user = MutableLiveData<User>()

    fun loadUser() {
        viewModelScope.launch {
            user.value = repo.getUser()
        }
    }
}
```

7.4 Redux (React Native / JS)

- Single global store
- State is read-only; you dispatch actions.
- A reducer decides new state.

Example (concept)

- State: { count: 0 }
- Action: { type: 'INCREMENT' }
- Reducer: returns { count: 1 }

Good when app has global complex state.